The folded pair-table build: a step probe

A diagnostic declared in `PLAN.md` before any run. It prices the steps of the build and decides nothing by itself.

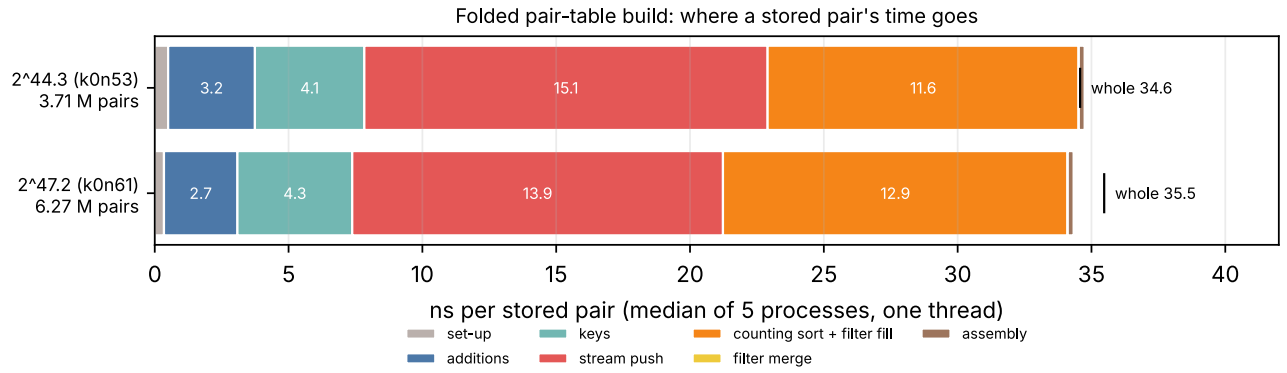
- **Binary:** `ic` (`ac8f3485…`, `--features scan-probes`) with this branch's `build_probe` timing points, which compile to nothing in a default build.
- **Rows:** suite `M1-T01` at `k0n53` (2^44.3) and `k0n61` (2^47.2), 5 processes each.
- **Isolation:** `isolated_bench --cpus 2` with `RAYON_NUM_THREADS=1`, on this session's container (Intel Xeon, AVX-512, 2 MiB L2 a core).
- **Builds:** each process builds the table 3 times (its repetitions), and the figures are per stored pair, per build.

Checks

- The steps sum to 98.6–98.9% (k0n53) and 99.2–99.3% (k0n61) of the build's wall time in every process. The plan's limit was 2%.
- The probe's whole build agrees with the report's `setup_phases_ns.build` to 0.02%.

Result: ns per stored pair, median [min, max] over 5 processes

step	k0n53 (3.71 M pairs)	k0n61 (6.27 M pairs)
set-up	0.50 [0.45, 0.56]	0.34 [0.32, 0.36]
row additions	3.24 [2.72, 3.27]	2.74 [2.62, 3.01]
row keys	4.09 [3.83, 4.33]	4.29 [4.00, 4.37]
stream push	**15.07** [12.73, 16.07]	**13.85** [13.10, 16.72]
counting sort + filter fill	**11.63** [10.10, 13.32]	**12.88** [11.65, 14.00]
filter merge, harvest	0, 0 (one thread; harvest off)	0, 0
assembly	0.22 [0.17, 0.24]	0.22 [0.18, 0.25]
whole	**34.58** [31.62, 38.05]	**35.48** [32.44, 37.79]



Reading

- **The build's time is in placing pairs, not in forming them.** Stream push plus the counting sort and filter fill take 26.7 and 26.7 ns of about 35, or 77% and 75% of the build. The additions and keys together take 7.3 and 7.0 ns.
- **That explains the earlier null result:** the twin-pairing A/B could at most move the 3 ns of additions.

Micro-benchmark of the two placing steps (`microbench/`)

A standalone copy of steps 1 and 2 ran on random keys at the same pair counts and bucket counts, pinned to one CPU, best of 7, and checked against the baseline's output. Spread between runs is about ±2 ns.

- **Floor:** reading and hashing a key costs about 1 ns, and storing it sequentially 2–4 ns.
- **Push variants:** 128 or 64 partitions, software write-combining (with or without non-temporal stores), one skewed flat buffer, and pre-faulted memory. None gains more than about 1–2 ns, which is within the spread.
- **Placement with a partition-local filter word:** this is the one large effect. Take the filter's word index from the hash's top bits, the bits that already choose the partition, so that each partition fills only its own about 16 KB of the filter. Placement drops from 8.5–12.6 to 5.0–5.7 ns per pair at both sizes. That would be about 4–6 ns of the build's 35: 15–22 ms of a 369 ms set-up at k0n53 (4–6%), and 25–38 ms of 1357 ms at k0n61 (2–3%), with set-ups the medians of this run. This holds only if the micro-benchmark carries over to the real build, which has not been measured.

- It is a table-format change. The filter word moves from the hash's low bits to its high bits. The GPU kernel (`gpu/ecc2k/pairtable.cuh`, `fold2k.cu`) and stored tables share today's layout.
- It is not made here.

Scope

A diagnostic on this host class, with no speed claim and no board row. Iso Lab is blocked while the CodeBuild quota reads 0.